

Full Project Proposal

Project Title

Design and Development of a Single-Hotel AI Receptionist Prototype for Booking Inquiry and Request Handling

Project Domain

Artificial Intelligence, Natural Language Processing, Full-Stack Web Development, and Hotel Service Automation

Project Idea

This project proposes the development of a **Single-Hotel AI Receptionist Prototype** that acts as a digital front-desk assistant for hotel guests. The system is intended to automate common receptionist tasks such as receiving booking-related inquiries, collecting guest request information, and supporting booking communication through a web-based interface.

The idea is based on the original concept of an AI-powered hotel receptionist that can communicate like a human receptionist, understand guest needs, and connect with hotel data.

In simple terms, the project is about creating a smart digital receptionist that can help hotel guests online, reduce repetitive front-desk work, and later grow into a more advanced AI assistant with conversational intelligence.

Background and Motivation

Hotel reception desks frequently manage repetitive guest interactions such as room inquiries, booking requests, pricing questions, and general assistance. In many hotels, these tasks are still handled manually, which can increase staff workload, slow down response time, and reduce service consistency.

This project is therefore motivated by two goals: first, to preserve the innovative AI receptionist concept; and second, to implement a technically complete prototype that can actually be developed, demonstrated, and extended.

Problem Statement

Traditional hotel front desks often face high call volumes, repetitive customer queries, and pressure to provide quick responses. This leads to delays, guest dissatisfaction, and higher dependency on human staff for routine operations. Existing manual workflows also make it difficult to digitize guest communication and track requests efficiently.

Although advanced AI receptionist systems can address these problems, they usually require complex technologies such as multi-channel integration, speech processing, local large language models, and scalable multi-tenant architecture. For an academic prototype, directly implementing such a broad system introduces high development risk and increases the chance of incomplete delivery.

There is therefore a need for a simplified but meaningful AI receptionist prototype that demonstrates the essential workflow of digital guest support: receiving a request, processing it through a backend, storing it in a database, and preparing the system for future intelligent conversational features.

Proposed Solution

The proposed solution is a **Single-Hotel AI Receptionist Prototype** built as a web-based full-stack application. A guest interacts with a digital receptionist interface on a hotel website and submits a booking-related inquiry or request. The backend receives that interaction, validates the data, stores it in a database, and returns an appropriate response to the user interface.

In the first version, the focus will be on a booking inquiry or request-handling workflow because it provides a realistic and technically demonstrable foundation. This means the first working milestone will function as a booking request handler inside the broader AI receptionist system.

The system is still considered an AI receptionist project because the architecture and long-term plan remain centered around intelligent interaction, automated assistance, and future conversational response generation. Later stages can extend the same prototype into a more natural text-based receptionist assistant and then, if feasible, into additional channels such as voice or messaging.

Main Objectives

The main objectives of this project are:

- To design and implement a **Single-Hotel AI Receptionist Prototype** that supports booking inquiry and request handling.
- To create a web-based user interface where guests can interact with the digital receptionist.
- To build a backend API that processes requests, performs validation, and stores records in a PostgreSQL database.

- To prepare the system for future AI-based conversational extensions.
- To build the project as a containerized, cloud-deployable, and GitHub Codespaces-compatible system.
- To align development with engineering-focused evaluation criteria such as testing, configuration management, logging, validation, deployment quality, and maintainability.

Specific Technical Objectives

The project will specifically aim to:

- Implement a working frontend-to-backend-to-database flow for guest request submission.
- Use environment-based configuration instead of hardcoded values.
- Add structured logging and meaningful error handling.
- Apply input validation and safe database interaction.
- Support containerized local and cloud-ready deployment.
- Create documentation that explains architecture, setup, and development workflow.

Business Objectives

From a business perspective, the project is intended to demonstrate how a hotel can reduce repetitive front-desk workload by automating common guest interactions. The prototype supports the idea that hotels can improve response speed, maintain service availability, and reduce operational friction through intelligent digital systems.

The business objectives are:

- To reduce repetitive receptionist tasks such as basic booking inquiry handling.
- To provide quicker and more consistent guest support.
- To create the foundation of a digital service that can operate with minimal manual intervention.
- To demonstrate a hotel technology product that could later evolve into a scalable commercial system.

Business Point of View

From a business point of view, hotels are service-oriented organizations where customer communication quality directly affects guest satisfaction. A delayed or inconsistent response can

reduce trust, while a fast and always-available digital assistant can improve customer experience. An AI receptionist system creates value in several ways:

- **Operational efficiency:** Staff spend less time on repetitive questions.
- **Availability:** The assistant can support guests beyond traditional front-desk hours.
- **Consistency:** Similar requests receive structured and standardized handling.
- **Scalability:** A successful single-hotel solution can later be extended to more hotels or more channels.
- **Digital transformation:** The project helps move hotel communication from manual handling to structured digital workflows.

The original project idea already framed the system as a product with SaaS potential for hotels. This proposal keeps that business vision, but the first phase is limited to a single-hotel prototype to make development more practical and demonstrable.

Scope of the Project

In Scope

The following items are included in the first version of the project:

- Single-hotel use case only.
- Web-based receptionist interface.
- Booking inquiry and request handling workflow.
- Frontend, backend, and PostgreSQL database integration.
- Containerization using Docker-based tools.
- Project setup suitable for GitHub Codespaces and reproducible development.
- Logging, validation, error handling, and basic testing support.
- Documentation of architecture and AI-assisted team workflow.

Out of Scope for Initial Prototype

The following items are intentionally postponed to future phases:

- Multi-hotel or multi-tenant SaaS support.

- WhatsApp integration through Twilio.
- Voice call handling.
- Speech-to-text and text-to-speech modules such as faster-whisper and Edge-TTS.
- Full autonomous booking execution.
- Advanced admin dashboard, subscriptions, and analytics platform.
- Heavy local model hosting as an initial requirement.

Target Users

The main target users of the system are:

- **Hotel guests**, who want quick answers or want to submit booking-related requests.
- **Hotel management or staff**, who benefit from structured digital request handling and reduced repetitive workload.

System Workflow

The simplified workflow of the project is as follows:

1. A guest opens the hotel receptionist web page.
2. The guest submits a booking-related request or inquiry.
3. The backend API receives the request.
4. The request is validated and processed.
5. The information is stored in a PostgreSQL database.
6. The system returns a response or confirmation to the user.
7. In future versions, an AI layer can generate smarter conversational replies and assist in interpreting user intent.

Development Approach

The project will follow an **Agile and iterative prototyping approach**. The original proposal already emphasized phased development, starting with core system elements and extending functionality over time. This proposal follows the same logic but in a smaller and more manageable form.

The development will be divided into practical stages:

- Stage 1: Project planning, repository setup, architecture design, and environment preparation.
- Stage 2: Backend API and database design.
- Stage 3: Frontend receptionist interface and request submission flow.
- Stage 4: Containerization, configuration management, and test setup.
- Stage 5: Refinement and preparation for future AI interaction features.

This staged method reduces risk, improves traceability, and makes sure the team produces a working system early instead of postponing all integration until the end.

Technologies to Be Used

The original documents mention Python, FastAPI, JavaScript, Next.js/React, PostgreSQL, and several AI-related components. Based on the reduced prototype scope, the following technology stack is proposed for the first implementation.

Layer	Technology	Purpose
Frontend	HTML/CSS/JavaScript or React/Next.js	Build the receptionist interface
Backend	Python with FastAPI	API development and request handling
Database	PostgreSQL	Store booking requests and guest-related records
Containerization	Docker / Docker Compose	Reproducible development and deployment
Version Control	Git / GitHub	Team collaboration and traceability
Dev Environment	GitHub Codespaces	Easy instructor testing and cloud-ready setup
Future AI Layer	LLM-based module	Natural language understanding and smart responses in later phases
Workflow Automation Support	n8n	n8n for notifications, routing, and helper automations via webhooks

Why These Technologies

These technologies were selected because they match the original project direction while still being practical for a semester prototype. FastAPI is suitable for lightweight API development, PostgreSQL supports structured data handling, and JavaScript or React provides a simple path for building an interactive receptionist interface. The use of GitHub, Docker, and Codespaces also supports

reproducibility, testing, and deployment quality. n8n is included only as a helper automation tool, not as a replacement for the main backend. The main application logic will still be implemented using Fast API and PostgreSQL, while n8n may support side workflows such as notifications or AI-assisted request handling.

Expected Features

The initial version of the project is expected to provide:

- A hotel receptionist web page.
- A booking inquiry or request submission action.
- Backend processing and database storage.
- Basic success and error responses.
- Configurable setup using environment variables.
- Containerized deployment structure.
- Readiness for future AI receptionist improvements.

Future Extensions

Once the base prototype is stable, the project can be extended with:

- conversational text chat,
- AI-based intent recognition,
- room availability lookup,
- automated booking assistance,
- admin dashboard features,
- WhatsApp support,
- and later voice interaction modules.

Expected Learning Outcomes

By completing this project, the team is expected to gain practical experience in:

- designing an AI-oriented system architecture,
- building full-stack web applications,

- implementing APIs and database integration,
- applying containerization and deployment practices,
- working with modern development tools and reproducible environments,
- and planning how conversational AI can be introduced in staged product development.

Deliverables

The expected project deliverables are:

- A project repository with source code and setup instructions.
- A working frontend interface for the AI receptionist prototype.
- A backend service connected to a PostgreSQL database.
- A working request-handling flow that stores booking-related data.
- Containerization and deployment-related configuration.
- A README file explaining system setup and team development process.
- Documentation for future extension into more advanced AI receptionist features.

Conclusion

This project presents a realistic and easy-to-understand academic prototype of an **AI-powered hotel receptionist**. It preserves the original innovative idea of a smart digital front desk assistant while reducing the scope to a single-hotel, technically manageable implementation.

The first working module focuses on booking inquiry and request handling because it provides a strong technical base and a clear demonstration of value. From there, the same prototype can evolve into a richer AI receptionist system with more natural conversation, smarter assistance, and broader hotel automation features.